

SpecTest-LLM: A Specification-Aware Multi-Agent Framework for Explainable and Feedback-Driven Test Case Generation

[Author Name(s)]
[Department / School]
[Institution Name]

City, Country

ail@institution.edu

Abstract—Automated test case generation is a critical challenge in software engineering, particularly when tests must exhibit high diversity, adequate edge-case coverage, and meaningful fault-detection capability. Although recent advances in large language models (LLMs) have demonstrated promising results in code generation and test synthesis, existing approaches predominantly rely on single-pass prompting, which lacks systematic coverage control, code-aware risk analysis, and structured output validation. This paper presents SpecTest-LLM, a specification-aware multi-agent framework that addresses these limitations through a decomposed, feedback-driven pipeline built on LangGraph. The system orchestrates five specialized agents—a Specification Extractor, a Code Analyzer, a Test Planner, a Test Generator, and a Feedback Analyzer—each operating on a shared state object. The Specification Extractor normalizes problem constraints and edge cases from natural language; the optional Code Analyzer identifies boundary conditions and structural risks; the Test Planner produces categorized coverage targets; the Test Generator emits explainable, categorized test cases in validated JSON; and the Feedback Analyzer detects weak coverage or malformed output and triggers targeted regeneration. A structured post-processing layer enforces per-category counts, normalizes labels, and validates outputs against Pydantic schemas. An empirical evaluation protocol is proposed comparing direct LLM prompting, multi-agent generation without refinement, and the full SpecTest-LLM pipeline across metrics including test validity rate, edge-case coverage, fault detection rate, redundancy rate, and parsing success rate. The results are expected to demonstrate that specification-aware multi-agent decomposition with feedback-driven refinement yields substantially improved test quality over single-pass baselines.

Index Terms—test case generation, large language models, multi-agent systems, LangGraph, specification extraction, feedback-driven refinement, software testing

I. INTRODUCTION

Software testing occupies a central role in ensuring the correctness, reliability, and robustness of modern software systems. Despite decades of research in automated testing, constructing high-quality test suites that exercise boundary conditions, corner cases, and fault-revealing inputs remains labor-intensive and domain-specific [8]. This challenge is especially acute in competitive programming and educational

contexts, where test suites must cover a broad spectrum of input categories to fairly assess candidate solutions.

The emergence of large language models (LLMs) has opened new possibilities for automating test case generation. Models such as GPT-4, Gemini, and Claude have demonstrated remarkable capabilities in understanding natural language problem descriptions, inferring program behavior, and generating syntactically plausible test inputs [10], [14]. However, deploying LLMs for test generation in a naive single-prompt fashion introduces several systematic deficiencies. First, a single prompt must simultaneously comprehend the specification, enumerate edge cases, impose coverage diversity, and produce well-formed structured output—tasks that compete for the model’s limited context and attention. Second, without an explicit feedback mechanism, a single-pass generation provides no means to detect or correct missing categories, malformed JSON, or redundant test cases. Third, when source code is available, single-pass prompting rarely integrates code-level risk signals such as loop boundary conditions or error-prone branching logic.

These limitations motivate a fundamentally different design paradigm: *specification-aware, feedback-driven multi-agent test generation*. By decomposing the generation task into a pipeline of specialized agents—each responsible for a well-defined subtask—the system can focus reasoning at each stage, accumulate richer intermediate representations, and iteratively refine outputs based on coverage analysis.

This paper introduces **SpecTest-LLM**, a multi-agent framework built on LangGraph [21] and LangChain [22] that generates explainable, categorized test cases from problem statements and optional source code. The key contributions of this work are as follows:

- 1) We propose SpecTest-LLM, a specification-aware multi-agent architecture that decomposes test case generation into five sequential agents operating on a shared state object.
- 2) We introduce a code-aware and specification-aware test planning mechanism that extracts input/output con-

straints, edge cases, and risky code behaviors before generation begins.

- 3) We design a feedback-driven refinement loop that detects weak coverage, malformed outputs, and missing test categories, then triggers targeted regeneration.
- 4) We provide an explainability layer that assigns each test case a category label and a natural-language explanation of its testing intent.
- 5) We present a rigorous empirical evaluation protocol comparing direct prompting, non-refined multi-agent generation, and the full SpecTest-LLM framework across eight evaluation metrics.

The remainder of this paper is organized as follows. Section II surveys related work. Section III presents the system methodology. Section IV describes the system architecture. Section V provides pseudocode for the core algorithms. Section VI details the experimental design. Section VII defines evaluation metrics. Section VIII reports and discusses results. Section IX presents a case study. Section X analyzes threats to validity. Section XI discusses limitations. Section XII outlines future work, and Section XIII concludes.

II. RELATED WORK

A. LLM-Based Code Generation

LLMs have achieved strong performance on standard code generation benchmarks. Zheng et al. [1] analyzed the reasoning strategies that enable LLMs to succeed on multi-turn code generation tasks, finding that structured intermediate reasoning substantially improves output correctness. Kuang et al. [3] examined training data optimization for code generation models, demonstrating that curated specification-aligned data improves generalization. Majumdar et al. [4] proposed a synthetic data pipeline to augment LLM coding capabilities, while Tian et al. [5] specifically addressed specification misunderstanding in LLMs, showing that specification-aligned prompting reduces generation errors. These findings motivate the specification extraction stage in SpecTest-LLM.

B. LLM-Based Test Case Generation

A growing body of work has examined the use of LLMs for automated test case generation. Wang et al. [10] presented a comprehensive benchmark of LLM test generation across multiple programming domains, reporting wide variance in validity and fault-detection capability. Li et al. [14] evaluated multiple LLMs as test case generators and proposed enhancement strategies based on structured prompting. Liu et al. [18] demonstrated that LLM-powered generation can detect subtle or “tricky” bugs when prompts are designed to elicit boundary-case reasoning. Yang et al. [19] introduced TestCase-Eval, a systematic evaluation framework for assessing fault coverage and exposure in algorithm problem test suites. Cao et al. [16] studied the reliability of LLM-generated test case generators for competition-level problems, identifying key failure modes including generator malfunction under stress inputs. Korrapolu et al. [20] and Sami et al. [6] both developed tooling for test scenario generation using LLMs, with emphasis on practical

deployment. Patel et al. [12] proposed a multi-modal context-aware framework for automated test generation that incorporates code structure alongside natural language descriptions.

C. Feedback-Driven and Iterative Generation

Cai et al. [2] proposed CodeContests-O, a system that powers LLMs through feedback-driven iterative test case generation for competitive programming, demonstrating that iterative refinement substantially improves test set quality. Chen et al. [15] developed a test case driven LLM debate framework in which multiple agents collectively improve code generation quality through iterative test-guided consensus. These works confirm that feedback loops are a highly effective design pattern for LLM-based generation, motivating the feedback refinement loop central to SpecTest-LLM.

D. Assertion Generation and Unit Test Generation

Siddiq et al. [8] conducted an empirical study of JUnit test generation using LLMs, identifying key failure modes including assertion hallucination and missing exception handling. Lops et al. [9] built a system for automated unit test generation using LLMs with template-based scaffolding. Zhang et al. [11] explored automated assertion generation via LLMs, showing that structured prompting significantly improves assertion accuracy. Li et al. [17] investigated the effect of injecting project-level and testing-specific knowledge into LLM-based unit test generation, reporting substantial improvements in branch coverage and assertion quality.

E. Multi-Agent LLM Systems for Software Engineering

Kumari [7] proposed an intelligent test automation framework using a multi-agent LLM architecture, demonstrating that role-specialized agents outperform monolithic prompting for complex testing tasks. Lee [13] explored GPT-based systems for code review in educational settings, highlighting the value of structured agent roles in pedagogical applications. The multi-agent paradigm has also been applied to code repair, documentation generation, and security analysis, consistently showing that task decomposition improves output reliability and coverage.

F. Research Gap Summary

Despite the growing literature, several important gaps remain. First, most LLM-based test generation systems use single-pass prompting without systematic coverage planning or category enforcement. Second, code-aware risk analysis—using available source code to identify boundary conditions and structural vulnerabilities—is rarely integrated into the test generation pipeline. Third, feedback-driven refinement specifically targeting test coverage completeness and output validity has not been extensively explored. Fourth, explainability of generated test cases, through category labeling and natural language justification, is largely absent. SpecTest-LLM addresses all four gaps through its multi-agent, specification-aware, feedback-driven architecture.

III. PROPOSED METHODOLOGY

A. Problem Formulation

Let P denote a problem statement in natural language, C an optional source code artifact, L a target programming language, n the number of desired test suite variants, and k the target number of test cases per category. The objective is to produce a set of test suites $\mathcal{T} = \{T_1, T_2, \dots, T_n\}$ where each suite T_i comprises categorized test cases:

$$T_i = \{(x_j, y_j, \ell_j, e_j)\}_{j=1}^m \quad (1)$$

Here x_j is the input, y_j is the expected output, $\ell_j \in \mathcal{L}$ is a category label from a predefined label set, and e_j is a natural language explanation of the test case's intent. The framework additionally produces a structured specification S , a code analysis A , and a test plan Π as intermediate artifacts.

B. Overview of SpecTest-LLM

SpecTest-LLM is a LangGraph-based multi-agent pipeline in which a directed graph encodes the dependencies between agents and controls the flow of a shared state object σ . The state σ accumulates all intermediate and final outputs as the workflow executes:

$$\sigma = (P, D, R, L, C, S, A, \Pi, T, F, \mathcal{I}, \mathcal{J}) \quad (2)$$

where D is an optional user description, R is an optional set of constraints, S is the extracted specification, A is the code analysis, Π is the test plan, T is the generated test suite set, F is the feedback verdict, $\mathcal{I}$ is the set of detected issues, and $\mathcal{J}$ is the final JSON report.

C. Agent Responsibilities

1) *Specification Extractor Agent*: The Specification Extractor receives P , D , R , and L and produces a structured specification S in JSON format. The specification encodes:

- Input format (type, range, dimensions)
- Output format and invariants
- Explicit and implied constraints
- Assumptions about valid input
- An enumeration of expected edge cases

The agent uses a Pydantic schema to validate the output, ensuring downstream agents receive a normalized, reliable representation of the problem.

2) *Code Analyzer Agent*: When source code C is provided, the Code Analyzer examines the code structure to identify:

- Loop structures and their boundary conditions
- Conditional branches and their implicit assumptions
- Identified risk patterns (e.g., off-by-one errors, integer overflow risks)
- Untested code paths and likely failure points

If C is absent, this agent is skipped and the system operates in specification-only mode. The code analysis A augments the test planning stage with code-level coverage signals.

3) *Test Planning Agent*: The Test Planner receives S , A , and any feedback issues $\mathcal{I}$ from prior iterations. It produces a test plan Π specifying:

- A set of test categories $\mathcal{L} = \{\ell_1, \ell_2, \dots, \ell_c\}$
- Target count k_i for each category ℓ_i , where $\sum_i k_i \approx n \cdot k$

The planner is designed to balance coverage across typical case categories (e.g., normal inputs, boundary values, invalid inputs, stress tests, special structure cases) while respecting any regeneration guidance in $\mathcal{I}$.

4) *Test Generator Agent*: The Test Generator receives S , Π , and a suite index $i \in \{1, \dots, n\}$. It produces a test suite T_i in JSON format, where each test case carries a category label, input, expected output, and explanation. Post-processing enforces per-category counts, normalizes labels against $\mathcal{L}$, strips markdown fences, rewrites unsupported expressions, evaluates simple string operations, caps oversized stress values, and validates the output against the Pydantic test case schema.

5) *Feedback Analyzer Agent*: The Feedback Analyzer receives S , Π , T , and $\mathcal{I}$, and produces a binary verdict $F \in \{\text{refine}, \text{finalize}\}$. It detects:

- Missing or underrepresented test categories
- Malformed or unparseable outputs
- Excessive test case redundancy
- Category count deviations exceeding a threshold δ

If $F = \text{refine}$, the system returns to the Test Planner with an updated issue list $\mathcal{I}$. If $F = \text{finalize}$, the system proceeds to report generation.

D. Feedback-Driven Refinement Loop

The refinement loop terminates when either $F = \text{finalize}$ or a maximum iteration count $\tau_{\max}$ is reached. The iterative process can be characterized as:

$$(\Pi^{(t+1)}, T^{(t+1)}) = \text{Refine}(\Pi^{(t)}, T^{(t)}, \mathcal{I}^{(t)}) \quad (3)$$

where t denotes the iteration index. This loop ensures that structural weaknesses identified in round t are corrected in round $t+1$ through targeted planning adjustments.

E. Structured Output Parsing and Validation

LLM outputs are inherently prone to formatting errors. SpecTest-LLM applies a multi-stage post-processing pipeline:

- 1) Strip markdown code fences (e.g., `` `json` ``)
- 2) Extract valid JSON blobs using regex-based detection
- 3) Rewrite unsupported JavaScript-style expressions (e.g., `.repeat(n) → * n`)
- 4) Evaluate safe string expressions (e.g., `"a" * 1000`)
- 5) Cap oversized string values to a maximum length $L_{\max}$
- 6) Validate parsed structure against Pydantic schemas
- 7) Enforce per-category test counts per the plan Π
- 8) Normalize category labels to the canonical label set $\mathcal{L}$

F. Explainability Layer

Each generated test case carries an `explanation` field populated by the Test Generator. This field provides a natural-language justification for why the test case was created, referencing the specification constraint or code risk it exercises. This design is consistent with the principle of explainable software engineering tools [7].

G. Report Generation and UI

Upon finalization, the Report Builder assembles the complete JSON report $\mathcal{J}$ containing the extracted specification, code analysis, test plan, all test suites, and the final feedback verdict. The Gradio-based UI [24] renders categorized test cases in a scrollable panel with per-suite copy buttons that export inputs in order. The full JSON report is displayed below the panel for downstream integration.

IV. SYSTEM ARCHITECTURE

Fig. 1 presents the overall system architecture of SpecTest-LLM. The architecture comprises six logical layers:

Layer 1 – User Interface: The Gradio-based web UI collects user inputs including the problem statement, optional description, constraints, source code, target language, and generation parameters (n, k) . It also renders the final test suites and JSON report.

Layer 2 – Python Orchestrator: A Python orchestration module manages state initialization, triggers the LangGraph workflow, and coordinates post-processing and report assembly.

Layer 3 – LangGraph Workflow: The directed state graph defines agent execution order, conditional routing (the `refine/finalize` branch), and shared state management. LangGraph [21] provides cycle support necessary for the feedback refinement loop.

Layer 4 – Agent Modules: Five specialized agents are implemented in `agents.py`, each with a dedicated prompt template, output parser, and error handler. Prompt templates are stored separately from orchestration logic to facilitate maintenance and reproducibility.

Layer 5 – Gemini Model Layer: Each agent is backed by a Gemini model instance configured in `llms.py`. Different model variants are assigned to agents based on task complexity: the Specification Extractor and Feedback Analyzer use `gemini-3-flash-preview`; the Test Planner uses `gemini-3.1-flash-lite-preview`; the Code Analyzer uses `gemini-2.5-flash`; and the Test Generator uses `gemini-2.5-flash-lite`.

Layer 6 – Schema and Validation Layer: Pydantic [25] schemas defined in `schemas.py` enforce the structural integrity of all intermediate and final outputs.

Table I summarizes the agents, their inputs, outputs, and assigned models.

TABLE I
AGENT SUMMARY: INPUTS, OUTPUTS, AND MODEL ASSIGNMENTS

Agent	Input	Output	Model
Spec Extractor	P, D, R, L	S (JSON spec)	gemini-3-flash-preview
Code Analyzer	C, L	A (analysis)	gemini-2.5-flash
Test Planner	$S, A, \mathcal{I}$	Π (plan)	gemini-3.1-flash-lite-preview
Test Generator	S, Π, i	T_i (suite)	gemini-2.5-flash-lite
Feedback Analyzer	$S, \Pi, T, \mathcal{I}$	$F \in \{\text{refine, finalize}\}$	gemini-3-flash-preview

Fig. 1: Overall architecture of SpecTest-LLM showing the six-layer system stack from the Gradio UI to the Pydantic schema validation layer. [Figure to be inserted]

Fig. 1. Overall architecture of SpecTest-LLM.

Fig. 2: LangGraph-based multi-agent workflow depicting the directed state graph, agent nodes, conditional routing, and shared state transitions. [Figure to be inserted]

Fig. 2. LangGraph-based multi-agent workflow.

Fig. 3: Feedback-driven refinement loop showing the detection of coverage deficiencies and the iterative replanning and regeneration cycle. [Figure to be inserted]

Fig. 3. Feedback-driven refinement loop.

Fig. 4: Evaluation protocol showing the dataset, baseline methods, execution engine, and metric computation pipeline. [Figure to be inserted]

Fig. 4. Evaluation protocol for SpecTest-LLM.

Fig. 5: UI and report generation flow from agent output to Gradio rendering and JSON export. [Figure to be inserted]

Fig. 5. UI and report generation flow.

V. ALGORITHMS

Algorithm 1 presents the top-level SpecTest-LLM workflow. Algorithm 2 describes the feedback-driven refinement procedure. Algorithm 3 defines the validity and coverage computation. Algorithm 4 specifies the fault detection evaluation procedure.

Algorithm 1 SpecTest-LLM Workflow

Require: Problem P , description D , constraints R , language L , code C , suites n , cases per category k

Ensure: Test suites $\mathcal{T}$, report $\mathcal{J}$

```
1: Initialize shared state  $\sigma \leftarrow (P, D, R, L, C, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$ 
2:  $S \leftarrow \text{SPECExtractor}(P, D, R, L)$ ; update  $\sigma.S \leftarrow S$ 
3: if  $C \neq \emptyset$  then
4:    $A \leftarrow \text{CODEANALYZER}(C, L)$ ; update  $\sigma.A \leftarrow A$ 
5: end if
6:  $t \leftarrow 0$ ;  $\mathcal{I} \leftarrow \emptyset$ 
7: repeat
8:    $\Pi \leftarrow \text{TESTPLANNER}(S, A, \mathcal{I})$ ; update  $\sigma.\Pi \leftarrow \Pi$ 
9:    $\mathcal{T} \leftarrow \emptyset$ 
10:  for  $i = 1$  to  $n$  do
11:     $T_i \leftarrow \text{TESTGENERATOR}(S, \Pi, i)$ 
12:     $T_i \leftarrow \text{POSTPROCESS}(T_i, \Pi)$ 
13:     $\mathcal{T} \leftarrow \mathcal{T} \cup \{T_i\}$ 
14:  end for
15:  update  $\sigma.T \leftarrow \mathcal{T}$ 
16:   $(F, \mathcal{I}) \leftarrow \text{FEEDBACKANALYZER}(S, \Pi, \mathcal{T}, \mathcal{I})$ 
17:   $t \leftarrow t + 1$ 
18: until  $F = \text{finalize}$  or  $t \geq \tau_{\max}$ 
19:  $\mathcal{J} \leftarrow \text{BUILDREPORT}(\sigma)$ 
20: return  $\mathcal{T}, \mathcal{J}$ 
```

Algorithm 2 Feedback-Driven Refinement (FEEDBACKANALYZER)

Require: Spec S , plan Π , test suites $\mathcal{T}$, prior issues $\mathcal{I}$, threshold δ

Ensure: Verdict $F \in \{\text{refine}, \text{finalize}\}$, updated issues $\mathcal{I}'$

```
1:  $\mathcal{I}' \leftarrow \emptyset$ 
2: for each category  $\ell_i \in \Pi.\mathcal{L}$  do
3:    $\text{actual}_i \leftarrow |\{t \in \mathcal{T} : t.\ell = \ell_i\}|$ 
4:   if  $|\text{actual}_i - \Pi.k_i| > \delta$  then
5:      $\mathcal{I}' \leftarrow \mathcal{I}' \cup \{(\ell_i, \text{"count deviation"}, \Pi.k_i - \text{actual}_i)\}$ 
6:   end if
7: end for
8:  $\text{parseErrors} \leftarrow \text{count malformed outputs in } \mathcal{T}$ 
9: if  $\text{parseErrors} > 0$  then
10:   $\mathcal{I}' \leftarrow \mathcal{I}' \cup \{(\text{"parse\_error"})\}$ 
11: end if
12:  $\text{covered} \leftarrow |\{\ell_i \in \Pi.\mathcal{L} : \text{actual}_i > 0\}|$ 
13: if  $|\mathcal{I}'| = 0$  and  $\text{covered} = |\Pi.\mathcal{L}|$  then
14:   return  $\text{finalize}, \mathcal{I}'$ 
15: else
16:   return  $\text{refine}, \mathcal{I}'$ 
17: end if
```

Algorithm 3 Test Validity and Coverage Computation

Require: Test suites $\mathcal{T}$, plan Π , execution results $\mathcal{E}$

Ensure: Validity rate VR, edge-case coverage EC, redundancy rate RR

```
1:  $\text{total} \leftarrow \sum_{T_i \in \mathcal{T}} |T_i|$ 
2:  $\text{valid} \leftarrow |\{t \in \mathcal{T} : \text{ISVALID}(t, \mathcal{E})\}|$ 
3:  $\text{VR} \leftarrow \text{valid}/\text{total}$ 
4:  $\text{targetCats} \leftarrow \Pi.\mathcal{L}$ 
5:  $\text{coveredCats} \leftarrow \{\ell \in \text{targetCats} : \exists t \in \mathcal{T}, t.\ell = \ell\}$ 
6:  $\text{EC} \leftarrow |\text{coveredCats}|/|\text{targetCats}|$ 
7:  $\text{dups} \leftarrow |\{(t_a, t_b) \in \mathcal{T}^2 : t_a \neq t_b \wedge \text{SIM}(t_a, t_b) > \theta\}|$ 
8:  $\text{RR} \leftarrow \text{dups}/\text{total}$ 
9: return VR, EC, RR
```

Algorithm 4 Fault Detection Evaluation

Require: Test suites $\mathcal{T}$, buggy solutions $\mathcal{B} = \{b_1, \dots, b_m\}$, timeout τ

Ensure: Fault detection rate FDR

```
1:  $\text{detected} \leftarrow 0$ 
2: for each buggy solution  $b_j \in \mathcal{B}$  do
3:    $\text{killed}_j \leftarrow \text{false}$ 
4:   for each test suite  $T_i \in \mathcal{T}$  do
5:     for each test case  $t \in T_i$  do
6:        $\text{out} \leftarrow \text{EXECUTE}(b_j, t.x, \tau)$ 
7:       if  $\text{out} \neq t.y$  or  $\text{out} = \text{TIMEOUT}$  then
8:          $\text{killed}_j \leftarrow \text{true}$ ; break
9:       end if
10:    end for
11:    if  $\text{killed}_j$  then
12:      break
13:    end if
14:  if  $\text{killed}_j$  then
15:     $\text{detected} \leftarrow \text{detected} + 1$ 
16:  end for
17:  $\text{FDR} \leftarrow \text{detected}/|\mathcal{B}|$ 
18: return FDR
```

VI. EXPERIMENTAL DESIGN

A. Dataset Construction

A curated benchmark of 25 programming problems is assembled spanning six categories: (1) arrays, (2) strings, (3) mathematics, (4) searching and sorting, (5) basic recursion and dynamic programming, and (6) basic graph problems. Problems are sourced from publicly available competitive programming repositories and educational platforms, selected to ensure a diverse range of input structures, constraint types, and edge-case patterns.

For each problem, the benchmark provides:

- A natural language problem statement and constraints
- One verified correct solution
- Two to three buggy solutions exhibiting distinct fault types

Bug types introduced in the buggy solutions include: wrong boundary condition, wrong loop range, missing edge case, incorrect comparison operator, duplicate handling issue, empty input issue, integer overflow risk, and timeout-inducing inefficiency. Table II summarizes the dataset distribution.

TABLE II
BENCHMARK DATASET DISTRIBUTION

Problem Category	# Problems	# Buggy Solutions
Arrays	5	10–15
Strings	4	8–12
Mathematics	4	8–12
Searching & Sorting	4	8–12
Recursion / DP	4	8–12
Graph Problems	4	8–12
Total	25	50–75

B. Baseline Methods

Three configurations are evaluated:

Baseline 1 – Direct LLM Prompting (DLP): A single prompt containing the problem statement and a request for test cases is issued to a Gemini model. No structured planning, code analysis, or feedback is applied.

Baseline 2 – Multi-Agent Without Feedback (MA-NF): The full specification extraction, code analysis, planning, and generation pipeline is executed, but the feedback refinement loop is disabled. The system runs exactly one generation pass.

Proposed – SpecTest-LLM (ST-LLM): The complete system with specification extraction, code analysis, test planning, structured post-processing, feedback analysis, and iterative refinement.

C. Evaluation Environment

All experiments are conducted using the Google Gemini API [23] via LangChain Google GenAI. Test case execution is performed in an isolated Python sandbox with a per-test timeout of $\tau = 5$ seconds. Execution infrastructure is implemented using the `subprocess` module with resource limits to prevent runaway processes.

D. Prompting Setup

Each agent uses a fixed prompt template stored in `agents.py`. Prompt temperature is set to 0.2 for specification extraction and feedback analysis (to favor consistency) and 0.7 for test generation (to promote diversity). The maximum refinement iterations are set to $\tau_{\max} = 3$.

VII. EVALUATION METRICS

A. Test Validity Rate (TVR)

The fraction of generated test cases that are syntactically valid, parseable, and produce deterministic expected outputs:

$$\text{TVR} = \frac{|\{t \in \mathcal{T} : \text{valid}(t)\}|}{|\mathcal{T}|} \quad (4)$$

B. Edge-Case Coverage (ECC)

The fraction of target test categories that are represented by at least one test case:

$$\text{ECC} = \frac{|\{\ell \in \Pi.\mathcal{L} : \exists t \in \mathcal{T}, t.\ell = \ell\}|}{|\Pi.\mathcal{L}|} \quad (5)$$

C. Fault Detection Rate (FDR)

The fraction of buggy solutions that are detected (killed) by the generated test suites:

$$\text{FDR} = \frac{|\{b \in \mathcal{B} : \exists t \in \mathcal{T}, \text{exec}(b, t) \neq t.y\}|}{|\mathcal{B}|} \quad (6)$$

D. Redundancy Rate (RR)

The fraction of test case pairs that are semantically similar (as measured by input string similarity above threshold $\theta = 0.9$):

$$\text{RR} = \frac{|\{(t_a, t_b) \in \mathcal{T}^2 : t_a \neq t_b \wedge \text{sim}(t_a, t_b) > \theta\}|}{|\mathcal{T}| \cdot (|\mathcal{T}| - 1)} \quad (7)$$

E. Parsing Success Rate (PSR)

The fraction of agent invocations that produce syntactically valid, schema-compliant JSON:

$$\text{PSR} = \frac{\text{valid JSON outputs}}{\text{total agent invocations}} \quad (8)$$

F. Refinement Gain (RG)

The improvement in edge-case coverage after the feedback refinement loop compared to the pre-refinement output:

$$\text{RG} = \text{ECC}_{\text{post}} - \text{ECC}_{\text{pre}} \quad (9)$$

G. Expected Output Availability (EOA)

The fraction of generated test cases that include a non-empty expected output:

$$\text{EOA} = \frac{|\{t \in \mathcal{T} : t.y \neq \emptyset\}|}{|\mathcal{T}|} \quad (10)$$

H. Execution Success Rate (ESR)

The fraction of test executions that complete without infrastructure failure (i.e., without sandbox crash, memory error, or encoding error):

$$\text{ESR} = \frac{\text{executions completed without infra failure}}{\text{total executions attempted}} \quad (11)$$

VIII. RESULTS AND DISCUSSION

Note: The quantitative evaluation will be completed after executing the proposed framework on the curated benchmark. The tables below provide the evaluation template used for reporting the final experimental results.

A. Overall Comparison

Table III presents the overall comparison between the three configurations.

TABLE III
OVERALL METRIC COMPARISON ACROSS BASELINE CONFIGURATIONS

Metric	DLP	MA-NF	ST-LLM	Gain
TVR (%)	–	–	–	–
ECC (%)	–	–	–	–
FDR (%)	–	–	–	–
RR (%)	–	–	–	–
PSR (%)	–	–	–	–
EOA (%)	–	–	–	–
ESR (%)	–	–	–	–
RG	–	–	–	–

DLP: Direct LLM Prompting; MA-NF: Multi-Agent No Feedback; ST-LLM: SpecTest-LLM

B. Category-Wise Edge-Case Coverage

Table IV provides a breakdown of edge-case coverage by problem category.

TABLE IV
CATEGORY-WISE EDGE-CASE COVERAGE (%)

Category	DLP	MA-NF	ST-LLM
Arrays	–	–	–
Strings	–	–	–
Mathematics	–	–	–
Searching & Sorting	–	–	–
Recursion / DP	–	–	–
Graph Problems	–	–	–

C. Fault Detection Comparison

Table V reports fault detection rates by bug type.

TABLE V
FAULT DETECTION RATE BY BUG TYPE (%)

Bug Type	DLP	MA-NF	ST-LLM
Wrong boundary condition	–	–	–
Wrong loop range	–	–	–
Missing edge case	–	–	–
Incorrect comparison	–	–	–
Duplicate handling	–	–	–
Empty input	–	–	–
Integer overflow	–	–	–
Inefficient logic	–	–	–

D. Refinement Gain Analysis

Table VI compares edge-case coverage before and after the feedback refinement loop in the ST-LLM configuration.

TABLE VI
EDGE-CASE COVERAGE BEFORE AND AFTER REFINEMENT (ST-LLM)

Category	Pre-Refinement	Post-Refinement	RG
Arrays	–	–	–
Strings	–	–	–
Mathematics	–	–	–
Searching & Sorting	–	–	–
Recursion / DP	–	–	–
Graph Problems	–	–	–
Average	–	–	–

E. Discussion

The design rationale for SpecTest-LLM anticipates several patterns in the experimental results. The Direct LLM Prompting baseline is expected to exhibit the lowest edge-case coverage because a single prompt must simultaneously reason about specification, coverage strategy, and output format, leading to uneven category distribution. The Multi-Agent Without Feedback configuration is expected to improve over DLP by virtue of staged reasoning and structured planning, but will likely exhibit residual coverage gaps due to the absence of corrective feedback. The full SpecTest-LLM system is expected to achieve the highest ECC and FDR values by iteratively closing coverage gaps detected by the Feedback Analyzer.

The redundancy rate is expected to decrease with SpecTest-LLM because explicit per-category targets discourage the model from generating near-duplicate inputs within the same category. The parsing success rate is expected to be substantially higher for both multi-agent configurations compared to direct prompting, owing to the structured post-processing pipeline and Pydantic validation layer.

IX. CASE STUDY

To concretely illustrate the behavior of SpecTest-LLM, we present a worked example using the *Two Sum* problem: given an integer array `nums` and an integer `target`, return the indices of the two numbers that add up to `target`. Assume exactly one valid answer exists.

1) Extracted Specification (*S*):

- Input: integer array of length $2 \leq n \leq 10^4$, integers in $[-10^9, 10^9]$; integer target in $[-10^9, 10^9]$
- Output: list of two distinct zero-based indices
- Constraints: exactly one solution guaranteed; cannot reuse the same element
- Edge cases: array of length 2, target achieved by first and last elements, negative integers, large values near integer bounds

2) *Code Analysis (A)*: A hash-map based solution is analyzed. The Code Analyzer identifies: a single-pass loop with $O(n)$ complexity, a dictionary lookup on the complement, and the risk of incorrect index tracking if the same value appears twice. The key risk flagged is: *early-exit may fail if the complement appears at index 0 and the falsy check is used.*

3) *Test Plan (II)*: The Test Planner produces six categories: (1) normal cases, (2) boundary size (length 2), (3) negative integers, (4) large values near integer limit, (5) target at extremes of the array, (6) duplicate values.

4) *Generated Test Cases*: A representative selection from suite T_1 includes:

- **Normal:** `nums=[2,7,11,15], target=9 → [0,1]; explanation: standard case verifying basic hash-map lookup.`
- **Boundary:** `nums=[1,3], target=4 → [0,1]; explanation: minimum array size.`
- **Negative:** `nums=[-3,-1,4,2], target=1 → [0,3]; explanation: tests handling of negative integers.`
- **Large values:** `nums=[1000000000,-1000000000], target=0 → [0,1]; explanation: near-integer-limit inputs.`
- **Duplicate values:** `nums=[3,3], target=6 → [0,1]; explanation: tests that duplicate elements with different indices are handled correctly.`

5) *Feedback Refinement*: In the first iteration, the Feedback Analyzer detects that the *large values* category is under-represented (one case vs. target of two). It returns $F = \text{refine with issue: ('large values', 'count deficit', 1)}$. The planner adjusts the target and the generator produces an additional large-value case. In the second iteration, all categories are adequately covered and $F = \text{finalize}$ is returned.

6) *Final Report (J)*: The final report contains the specification JSON, code analysis, test plan, all n test suites, and the refinement history, suitable for integration into an automated grading or CI/CD pipeline.

X. THREATS TO VALIDITY

A. Internal Validity

The primary internal threat is LLM nondeterminism. Even with low temperature settings, repeated invocations of the same prompt may yield different outputs, introducing variance in measured metrics. This is mitigated by running each configuration three times and reporting mean values with standard deviation.

B. External Validity

The benchmark comprises 25 problems spanning six categories. While this covers a representative range of algorithmic problem types, generalization to larger, more complex problems (e.g., system design, concurrent programs) or other domains (e.g., web applications, ML pipelines) cannot be assumed. Future work will expand the benchmark.

C. Construct Validity

The evaluation metrics, particularly edge-case coverage and redundancy rate, require operational definitions. The threshold θ for semantic similarity in redundancy computation and the deviation threshold δ in the feedback loop are hyperparameters whose values affect reported results. Sensitivity analyses are planned to assess the impact of these thresholds.

D. Prompt Sensitivity

LLM performance is known to be sensitive to prompt phrasing [5]. The prompt templates used in SpecTest-LLM were developed through iterative refinement, but alternative phrasings may yield different performance levels. A prompt ablation study is planned as part of future work.

E. Dataset Construction Bias

Buggy solutions are constructed by introducing specific fault types into correct solutions. This controlled fault injection may not fully reflect the distribution of bugs found in real-world code. Evaluation on naturally occurring buggy submissions from online judges is planned.

XI. LIMITATIONS

SpecTest-LLM has several current limitations that are acknowledged as targets for future development.

No runtime execution engine in the prototype: The current implementation generates test cases and expected outputs but does not yet integrate a full runtime execution engine for automated verdict computation. The execution-based evaluation described in this paper is part of the proposed evaluation protocol rather than the deployed system.

Limited language support: The system currently targets Python and supports a limited set of other languages. Extending to compiled languages such as C++ and Java requires additional sandbox infrastructure.

Expected output reliability: For problems with complex output structures (e.g., multiple valid orderings, floating-point outputs), the LLM-generated expected outputs may be incorrect or underspecified. An execution-based expected output verifier is planned.

JSON compliance variability: Despite the post-processing pipeline, highly complex generation requests occasionally produce outputs that cannot be fully recovered. The parsing success rate, while high, is not 100%.

Stress test truncation: Very large stress inputs may be truncated to remain within the model’s context window and to keep the JSON report manageable. This may reduce coverage of true stress cases.

API latency and cost: Each run of SpecTest-LLM with refinement may invoke five to fifteen Gemini API calls depending on the number of refinement iterations and suite variants. This incurs latency of 10–60 seconds per problem and non-trivial API costs at scale.

XII. FUTURE WORK

Several directions extend the current work:

Runtime execution engine: Integrating a sandboxed execution engine (e.g., using Docker containers with resource limits) will enable verdict computation, expected output verification, and fault detection without human intervention.

Multi-language sandbox: Support for C++, Java, JavaScript, and other languages via language-specific execution environments will substantially expand applicability.

Mutation testing integration: Replacing manual buggy solution construction with automated mutation operators will enable scalable fault detection evaluation and more systematic measurement of test suite strength.

CI/CD and online judge integration: SpecTest-LLM can be integrated as a plugin in CI/CD pipelines or as a test generator for online programming judges, automating the evaluation of student submissions.

Retrieval-augmented generation (RAG): Incorporating a retrieval mechanism over past problem-test pairs may improve coverage by providing the planner with examples of effective test strategies for similar problems [17].

Fine-tuned test generation model: A domain-specific fine-tuned model trained on a corpus of high-quality test suites may outperform general-purpose Gemini models for specific problem categories.

Human-in-the-loop review: An interactive mode where human reviewers can inspect, modify, and approve generated test cases before finalization would improve reliability in high-stakes contexts.

Larger benchmark evaluation: Expanding the benchmark to 100+ problems from competitive programming platforms (e.g., Codeforces, LeetCode) will provide more robust empirical evidence.

XIII. CONCLUSION

This paper presented SpecTest-LLM, a specification-aware multi-agent framework for explainable and feedback-driven test case generation. By decomposing the generation task into five specialized agents—Specification Extractor, Code Analyzer, Test Planner, Test Generator, and Feedback Analyzer—orchestrated through a LangGraph-based directed state graph, the system achieves systematic coverage planning, code-aware risk analysis, structured output validation, and iterative refinement that are individually absent from single-pass LLM prompting approaches.

The key novelty of SpecTest-LLM lies in the integration of (1) specification-aware test planning that translates natural language constraints into categorized coverage targets, (2) a feedback-driven refinement loop that detects and corrects coverage deficiencies and parsing failures across iterations, and (3) an explainability layer that labels each test case by category and provides a natural-language justification for its testing intent.

An empirical evaluation protocol comparing direct LLM prompting, non-refined multi-agent generation, and the full SpecTest-LLM system across eight metrics is proposed and expected to demonstrate the advantages of the proposed architecture. The system is designed to be extensible, supporting future integration of runtime execution engines, multi-language sandboxes, mutation testing, and retrieval-augmented generation.

SpecTest-LLM represents a step toward principled, reliable, and interpretable automated test generation for algorithmic software, with broad applicability in educational assessment,

competitive programming, and industrial software quality assurance.

REFERENCES

- [1] K. Zheng et al., “What makes large language models reason in (multi-turn) code generation?,” *arXiv preprint*, 2024.
- [2] J. Cai et al., “CodeContests-O: Powering LLMs via feedback-driven iterative test case generation,” *arXiv preprint*, 2025.
- [3] S. Kuang et al., “On the effectiveness of training data optimization for LLM-based code generation,” *arXiv preprint*, 2025.
- [4] Majumdar et al., “Increasing LLM coding capabilities through a synthetic data pipeline,” *arXiv preprint*, 2024.
- [5] Tian et al., “Fixing large language models’ specification misunderstanding for better code generation,” *arXiv preprint*, 2024.
- [6] A. M. Sami et al., “A tool for test case scenarios generation using large language models,” in *Proc. [Conference]*, 2024.
- [7] P. Kumari, “Intelligent test automation: A multi-agent LLM framework,” *arXiv preprint*, 2025.
- [8] M. L. Siddiq et al., “Using large language models to generate JUnit tests: An empirical study,” in *Proc. [Conference]*, 2024.
- [9] A. Lops et al., “A system for automated unit test generation using large language models,” in *Proc. [Conference]*, 2024.
- [10] Wang et al., “Benchmarking large language models for test case generation,” *arXiv preprint*, 2025.
- [11] Q. Zhang et al., “Exploring automated assertion generation via large language models,” *arXiv preprint*, 2025.
- [12] P. K. Patel et al., “LLM-driven automated test case generation: A multi-modal context-aware framework,” in *Proc. [Conference]*, 2022.
- [13] D.-K. Lee, “A GPT-based code review system for programming language learning,” *[Journal]*, 2024.
- [14] K. Li et al., “Large language models as test case generators: Performance evaluation and enhancement,” *arXiv preprint*, 2024.
- [15] Chen et al., “Towards collective intelligence of LLMs via test case driven LLM debate for code generation,” *arXiv preprint*, 2025.
- [16] Y. Cao et al., “Can LLMs generate reliable test case generators? A study on competition-level programming problems,” *arXiv preprint*, 2025.
- [17] A. Li et al., “Knowledge matters: Injecting project and testing knowledge into LLM-based unit test generation,” *arXiv preprint*, 2026.
- [18] K. Liu et al., “LLM-powered test case generation for detecting tricky bugs,” in *Proc. [Conference]*, 2024.
- [19] Yang et al., “Can LLMs generate high-quality test cases for algorithm problems? TestCase-Eval: A systematic evaluation of fault coverage and exposure,” *arXiv preprint*, 2025.
- [20] Korrapolu et al., “A tool for test case scenarios generation using large language model,” in *Proc. [Conference]*, 2025.
- [21] LangChain AI, “LangGraph documentation,” [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [22] LangChain AI, “LangChain documentation,” [Online]. Available: <https://python.langchain.com/>
- [23] Google, “Gemini API documentation,” [Online]. Available: <https://ai.google.dev/>
- [24] Gradio Team, “Gradio documentation,” [Online]. Available: <https://www.gradio.app/>
- [25] Pydantic Team, “Pydantic documentation,” [Online]. Available: <https://docs.pydantic.dev/>